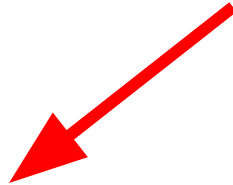


Chapter I

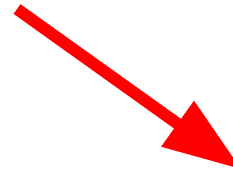
Motivation for Studying Data Structure

Program efficient ?

(A set of instructions to
perform certain task)



Data Structures
(Representation
of the data that
needs to be used
in the program)



Algorithm

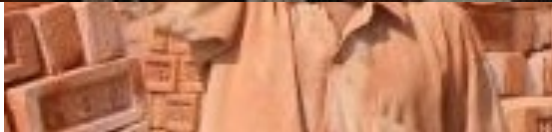
(Representation
of a solution to
the program)

+

both have to be efficient

An Example

- Building a house



- Bricks: data/ input, House: output
- Temporary structures to keep bricks: Data structure
 - Carrying bricks: Optimized movement of data

Why study data structure ?

- Proper usage of storage area

IEEE-754 standards integer/ float 4 bytes

Say, we have to store 1 million float values,
this will take $1,000,000 \times 4 \text{ bytes} = \mathbf{4 \text{ MB}}$

A square matrix of million X million float
values will take, $10^6 \times 10^6 \times 4 \text{ bytes} = \mathbf{4 \text{ TB}}$,
.....hence **data can not fit into memory** (if
sufficient resources are not available) and
program becomes slow

Why study data structure ? (contd.)

- Efficient manipulation of data
- Help reducing the time taken by algorithm
--- this is the **time complexity** of the algorithm.

Different data structures can be used in different versions of an algorithm and **time complexity may vary** based on that.

Time complexity



Have to catch a train at 6:00 p.m.

- Shorter eating time at 4:00 p.m. first can not
 Maximize time taken to get prepared
 (age of all five) 45 for the most
 inefficient 60 should be the time
 Researcher's "eat 15-min station".

Time complexity

Worst case run-time: for worst possible input sequence (upper bound)

→

19	23	15	8	31	5
----	----	----	---	----	---

Average case run-time: for 'average' input. All inputs of given size are equally likely

→

8	23	15	5	31	19
---	----	----	---	----	----

Best case run-time: for optimal input.

→

5	23	15	8	31	19
---	----	----	---	----	----

Time complexity

Worst case time-complexity is denoted by $T(n) = O(\dots) \rightarrow$ Big-Oh (means order of..)

Definition of Big-Oh notation:

Given two non-negative functions $f(n)$ and $g(n)$ there exists an integer n_0 and a constant $c > 0$, such that $\forall$ integers $n \geq n_0$, $|f(n)| \leq c \cdot |g(n)|$, then $f(n)$ is $O(g(n))$

worst case (upper bound)



Time complexity

Best case time-complexity is denoted by

$T(n) = \Omega(\dots)$ → Big-Omega

Definition of Big-Omega notation:

Given two non-negative functions $f(n)$ and $g(n)$ there exists an integer n_0 and a constant $c > 0$, such that $\forall$ integers $n \geq n_0$, $|f(n)| \geq c \cdot |g(n)|$, then $f(n)$ is $\Omega(g(n))$

best case (lower bound)



Time complexity

Combining Big-Oh and Big-Omega,

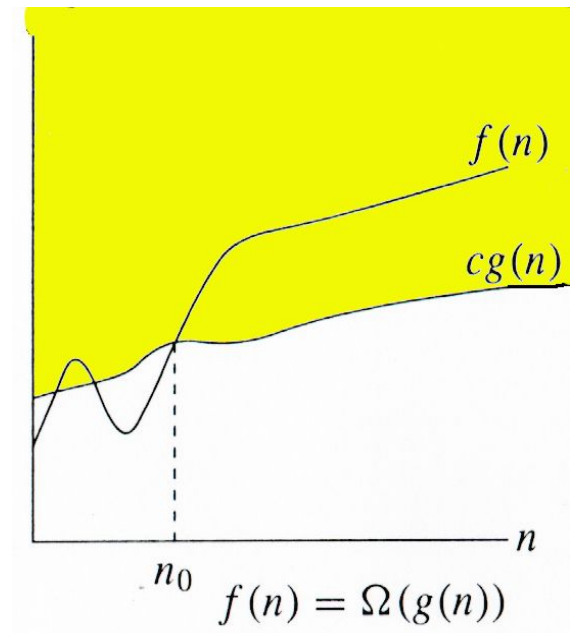
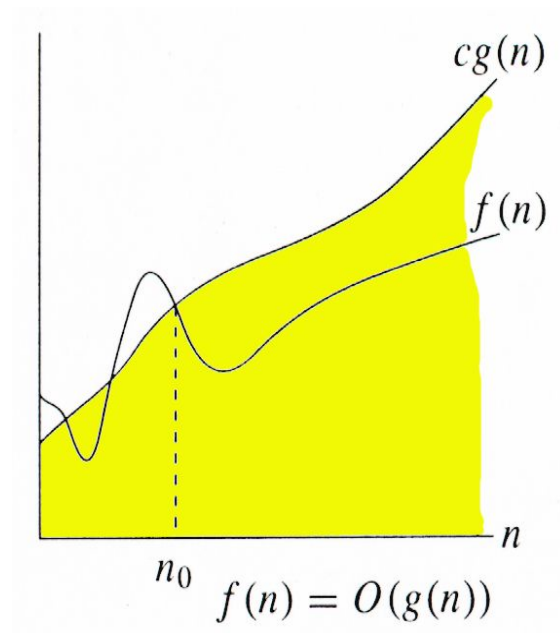
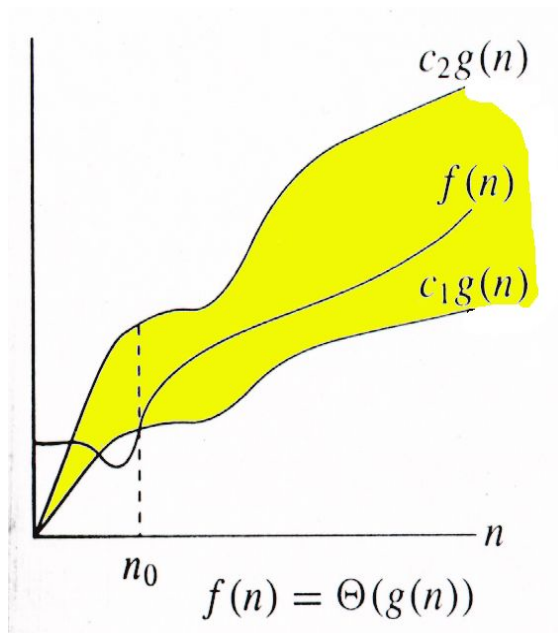
The diagram illustrates the combination of Big-Oh and Big-Omega to form Big-Theta. It features two overlapping ovals. The left oval, outlined in dark green, contains the expression $k_1 \cdot |g(n)| \geq$. The right oval, outlined in light blue, contains the expression $k_2 \cdot |g(n)|$. The intersection of these two ovals contains the function $f(n)$. A dark green arrow points from the left oval to the text 'Big-Oh', and a light blue arrow points from the right oval to the text 'Big-Omega'. Below the intersection, the variable n is centered. The entire diagram is set against a white background.

$$k_1 \cdot |g(n)| \geq f(n) \geq k_2 \cdot |g(n)|$$

Big-Oh n Big-Omega

....is called Big-Theta, $f = \Theta(g)$

Relations Between Θ , O , Ω



Time complexity

$O(f)$: Function grows **no faster** than f
(slower OR equal)

$\Omega(f)$: Function grows **no slower** than f
(faster OR equal)

$\Theta(f)$: Functions grows at the same rate as f

$o(f) = O(f) - \Theta(f)$ /* loosely bounds from top */

Little-Oh : Function grows slower than f

$\omega(f) = \Omega(f) - \Theta(f)$ /* loosely bounds from bottom */

Little-Omega : Function grows faster than f

Time complexity

Constant time - $O(1)$: Accessing elements of an array (independent on input size)

Double logarithmic time – $O(\log_2(\log_2 n))$:
(eg. interpolation search)

Logarithmic time – $O(\log_2 n)$: Loops with iterative halving (eg. Binary search)

Linear time - $O(n)$: Searching a number sequentially from a list (dependent linearly on the size of the input sequence)

Loglinear time – $O(n \cdot \log_2 n)$: two nested loops – one with linear iteration and the other with iterative halving. In which order? does not matter (eg. sorting methods such as merge sort, quick sort)

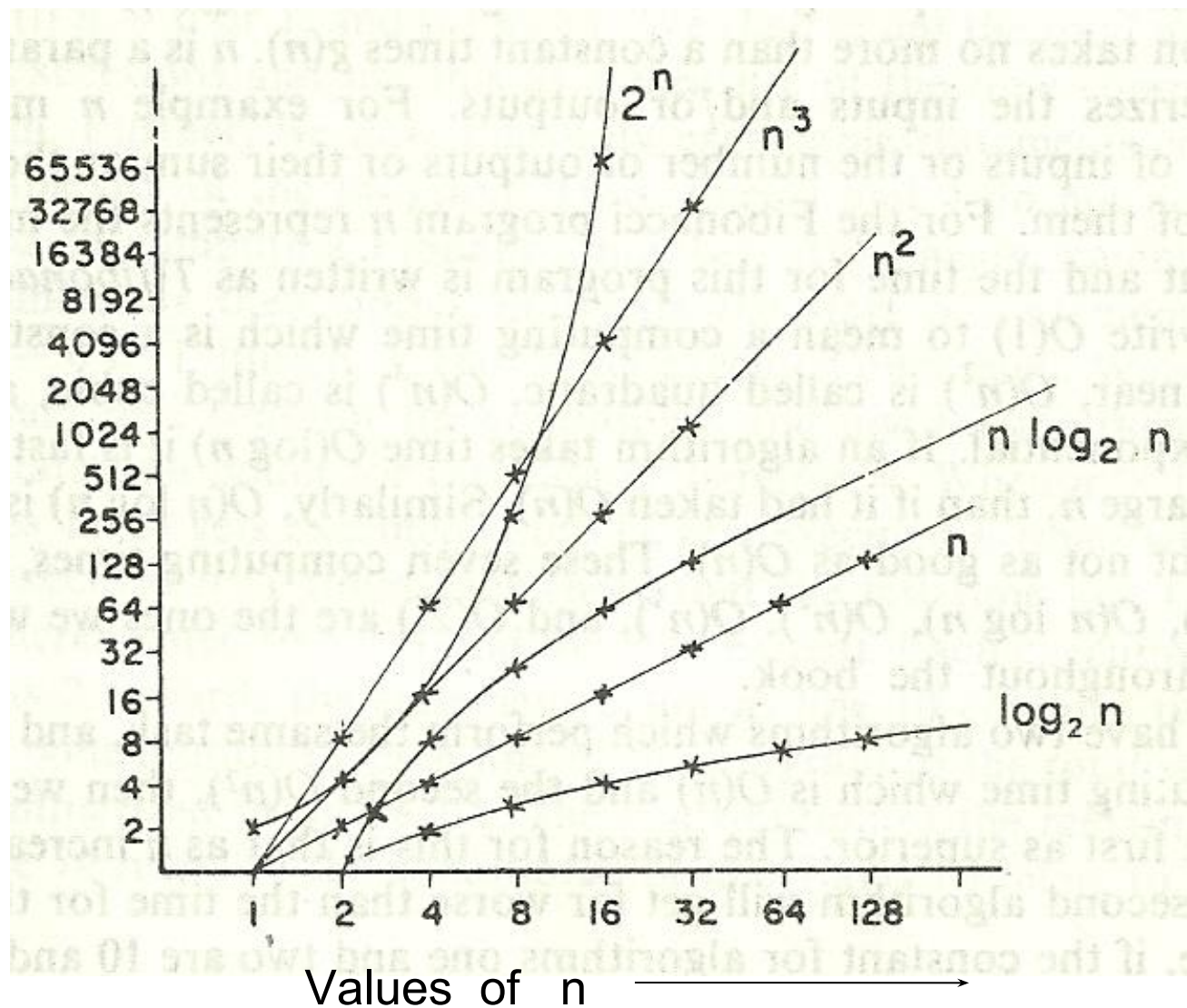
Quadratic time – $O(n^2)$: two nested loops with linear iteration (eg. naïve sorting methods such as bubble sort)

Polynomial time – $O(n^k)$

Exponential time – $O(c^n)$ / $O(2^n)$: generating all possible combinations (eg. Tower of Hanoi)

Factorial time – $O(n!)$: generating all unrestricted permutations

Comparing time complexities



Infinite Asymptotics

$T(n)$ needs to be judged based on the rate of growth

Say, for $T(n) = 4.n^2 - 8.n + 2$

n^2 will dominate over n as the value of n (size of the input) increases. So, $T(n)$ will depend on the term with n^2
By, asymptotic notation we can say that, $T(n) = O(n^2)$

What are the simplified Big-Oh / the orders of :

a. $T(n) = 6.n^3 - 5.n + 2$

b. $T(n) = 9.\log_2 n + 5.(\log_2 n)^3 + 2.n^3 + 3.n^2$

c. $T(n) = 4.2^n + n^{100} + 92.n^4 + 86$

Properties

- $O(f(n) + g(n)) = O(\max(f(n), g(n)))$
- if $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then,
 $f(n) = O(h(n))$ (transitivity)
- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

Data Types vs. Abstract Data Types

- ADT: Mathematical model for certain class of data structure.
 - Usually theoretical
 - ADT is defined by the operations of a data structure.
- Data types: Different types of data used in programming languages.
 - Usually used in programming
 - Defines scope of operations

How to write pseudo-code

- First few lines:

/* Name of the program */

Input: in writing

Output: in writing

- Loops:

for all <variable> **from** <initialized value> **to** <end value> **by** <step>
 statement
endfor

repeat
 statement
until <condition>

- Assignment:

$i \leftarrow \text{count}$ **/* i= count */**

- Begin-End:

begin
 statements
end

How to write pseudo-code

- Control statements:

```
[ if <condition> then do  
    statement  
else  
    statement  
endif
```

- Case statements:

```
[ case condition of  
    condition 1 : statement  
    condition 2 : statement  
    condition 3 : statement  
    condition 4 : statement  
endcase
```

Chapter II

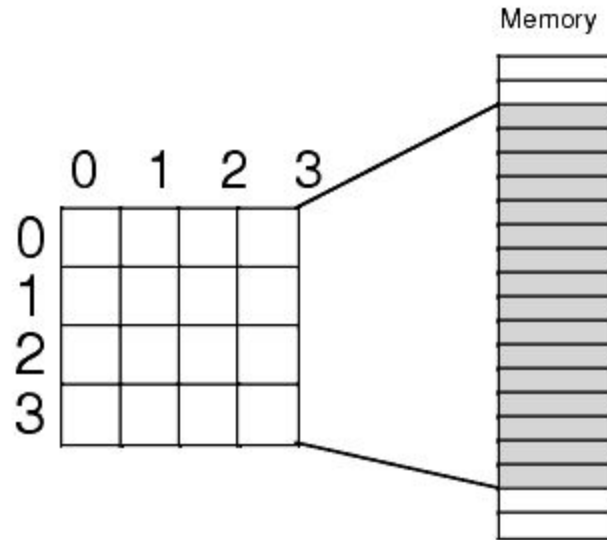
Linear Data Structures

Linear Data Structure

- Lists – Arrays, Linked lists
- Stack (LIFO)
- Queue (FIFO)

Arrays

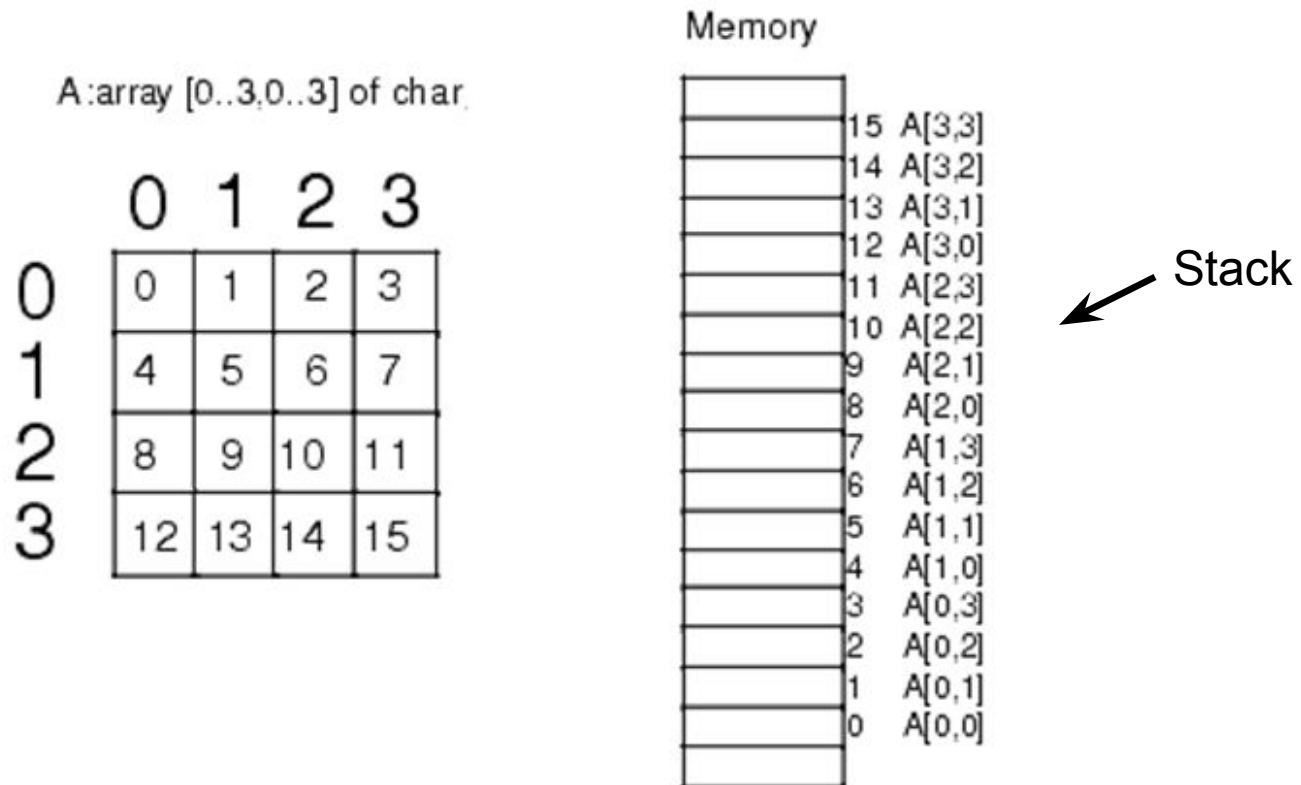
- Arrays can be multi-dimensional in structure, but memory is linear



- A multi-dimension to single dimension structure conversion is needed for storing the data in memory
- Two ways to do it –
 - i. Row-major ordering
 - ii. Column-major ordering

Arrays

- **Row-major ordering** –
Elements are stored by accessing one row at a time



Most HLLs like Pascal, C/C++, Java use this method

Arrays

- **Row-major ordering –**

2012 +4 +4 +4

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Say, we have a base address... can we find the address of any element (say 2,3) from the 2-d array?

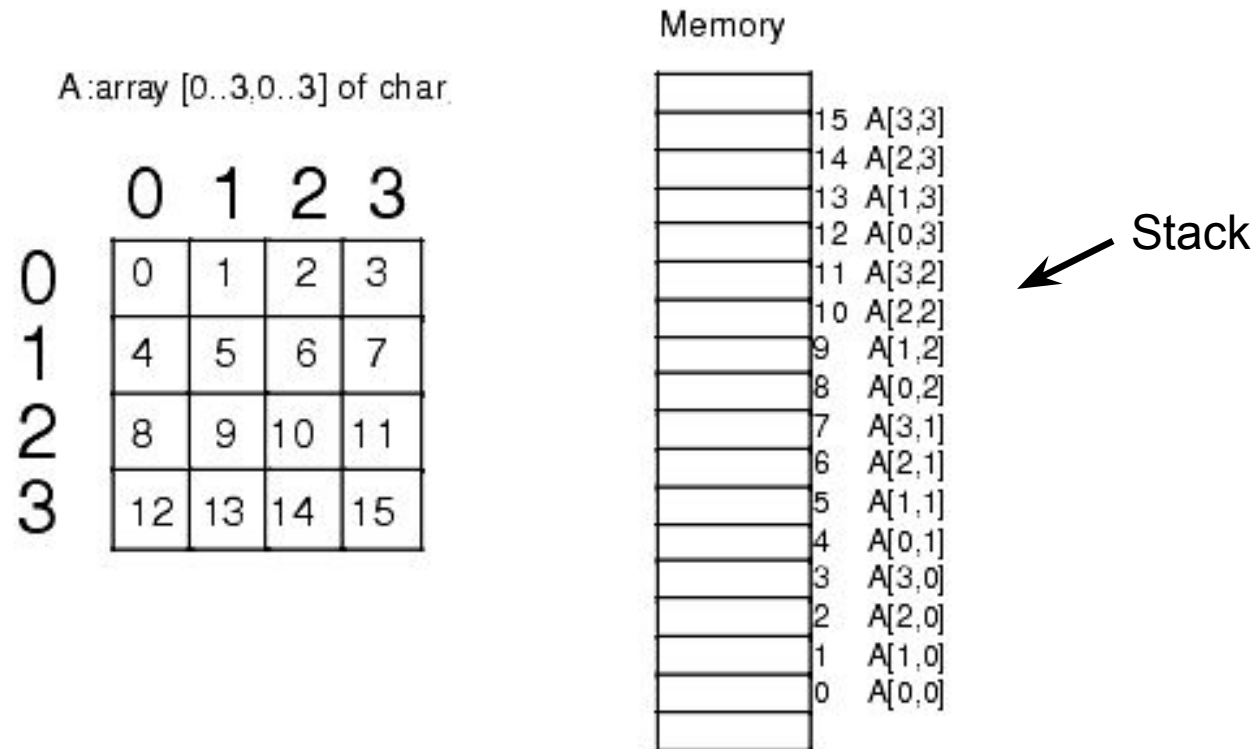
element_address

= base_add + (row_index * num_of_columns + column_index) * element_size

= 2012 + (2 X 4 + 3) X 4 = 2012 + 44 = 2056

Arrays

- **Column-major ordering** –
Elements are stored by accessing one column at a time



One a few rare HLLs like FORTRAN and MATLAB

Arrays

- **Column-major ordering –**

2012 +4 +4 +4

0	4	8	12	1	5	9	13	2	6	10	14	3	7	11	15
---	---	---	----	---	---	---	----	---	---	----	----	---	---	----	----

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Say, we have a base address... can we find the address of any element (say 2,3) from the 2-d array?

element_address

$$\begin{aligned} &= \text{base_add} + (\text{column_index} * \text{num_of_rows} + \text{row_index}) * \text{element_size} \\ &= 2012 + (3 * 4 + 2) * 4 = 2012 + 56 = 2068 \end{aligned}$$

Sparse Matrix

- How sparse is “sparse” enough –

	col 1	col 2	col 3		col 1	col 2	col 3	col 4	col 5	col 6
row 1	-27,	3,	4		15,	0,	0,	22,	0,	-15
row 2	6,	82,	-0.3		0,	11,	3,	0,	0,	0
row 3	109,	-64,	4		0,	0,	0,	-6,	0,	0
row 4	.12,	8,	9		0,	0,	0,	0,	0,	0
row 5	3.4,	36,	27		91,	0,	0,	0,	0,	0
				row 6	0,	0,	28,	0,	0,	0

Existence of non-zero elements - a lot less than maximum possible (n^2)

Sparse Representation

- Triple format –

0-th triple
denotes
info about
the matrix

of
rows

of
columns

of non
zero
elements

	col 1	col 2	col 3	col 4	col 5	col 6
row 1	15,	0,	0,	22,	0,	-15
row 2	0,	11,	3,	0,	0,	0
row 3	0,	0,	0,	-6,	0,	0
row 4	0,	0,	0,	0,	0,	0
row 5	91,	0,	0,	0,	0,	0
row 6	0,	0,	28,	0,	0,	0

	1,	2,	3,
A[0,	6,	6,	8
[1,	1,	1,	15
[2,	1,	4,	22
[3,	1,	6,	-15
[4,	2,	2,	11
[5,	2,	3,	3
[6,	3,	4,	-6
[7,	5,	1,	91
[8,	6,	3,	28

Other than 0-th
triple, rest
describe the
non-zero
elements

Triple is a tuple with three attributes

Sparse Representation

- Easy to manipulate –

Transpose of a matrix (row and column indices are swapped and put in a sorted order)

	1,	2,	3
A[0,	6,	6,	8
[1,	1,	1,	15
[2,	1,	4,	22
[3,	1,	6,	-15
[4,	2,	2,	11
[5,	2,	3,	3
[6,	3,	4,	-6
[7,	5,	1,	91
[8,	6,	3,	28

	1,	2,	3
B[0,	6,	6,	8
[1,	1,	1,	15
[2,	1,	5,	91
[3,	2,	2,	11
[4,	3,	2,	3
[5,	3,	6,	28
[6,	4,	1,	22
[7,	4,	3,	-6
[8,	6,	1,	-15

Triple format is efficient if $3 \cdot \text{NNZ} + 3 < N \cdot M$, where NNZ is number of non-zero elements, and the dimension of the matrix is $M \times N$

Polynomial Addition Using Array

- Use one global array to store all polynomials

$$A(X) = 2.X^{1000} + 1$$

$$B(X) = X^4 + 10.X^3 + 3.X^2 + 1$$

	<i>starta</i>	<i>finisha</i>	<i>startb</i>		<i>finishb</i>	<i>avail</i>
	↓	↓	↓		↓	↓
<i>coeff</i>	2	1	1	10	3	1
<i>exp</i>	1000	0	4	3	2	0
	0	1	2	3	4	5
						6

```

/* Program to add multiple polynomials */

#include<stdio.h>

/* Global structure that holds the polynomial terms
members store coefficients and exponents */

typedef struct
{
    float coeff;    // stores co-efficients of the terms
    int exp;        // stores exponents of the terms
}poly;

poly term[100];    //storing all the polynomials in one array of structure

int main(void)
{
    int i=0, j=0, k, m, numOfPoly, numOfTerms;
    printf("How many polynomials you want to add? ");
    scanf("%d", &numOfPoly);
    printf("\n");
    while(numOfPoly)
    {
        printf("How many terms?");
        scanf("%d", &numOfTerms);
        printf("\nEnter the co-eff and exps\n");
        k = 0;    //counting terms in polynomial starts
        while(numOfTerms)
        {
            scanf("%f %d", &term[i].coeff, &term[i].exp);
            i++;    //overall count of terms
            k++;    //count of terms for this polynomial only
            numOfTerms--;
        }
        j++;    //counting the polynomial
        k--;    //adjusting term count in polynomial
        printf("%d-th term is: ", j);
        for(m=i-1;m>=i-k-1;m--)    //printing only added terms for new polynomial
        {
            printf("%.2fx^%d ", term[m].coeff, term[m].exp);
            if(m>i-k-1) printf(" + ");
        }
        printf("\n");
        numOfPoly--;
    }
}

```

```

    i--; //adjusting total number of terms stored in array
    for(j=0; j<i; j++)
    {
        for(k=j+1; k<=i; k++)
        {
            if((term[j].coeff > 0) && (term[k].coeff > 0) && (term[j].exp == term[k].exp))
            {
                term[j].coeff += term[k].coeff;
                term[k].coeff = 0;
            }
        }
    }
    printf("\nThe added polynomial is: ");
    for(j=0; j<=i; j++)
    {
        if(term[j].coeff > 0)
        {
            printf("+ %.2fx^%d ", term[j].coeff, term[j].exp);
        }
    }
    return 0;
}

```

\$./polyAdd.exe

How many polynomials you want to add? 4

How many terms?3

Enter the co-eff and exps

2

3

3

4

4

5

1-th term is: $4.00x^5 + 3.00x^4 + 2.00x^3$

How many terms?2

Enter the co-eff and exps

2

5

3

4

2-th term is: $3.00x^4 + 2.00x^5$

How many terms?4

Enter the co-eff and exps

1

134

2

6

3

5

65

3

3-th term is: $65.00x^3 + 3.00x^5 + 2.00x^6 + 1.00x^{134}$

How many terms?2

Enter the co-eff and exps

2

4

3

5

4-th term is: $3.00x^5 + 2.00x^4$

The added polynomial is: $+ 67.00x^3 + 8.00x^4 + 12.00x^5 + 1.00x^{134} + 2.00x^6$